项目文件说明

广告素材合规审核系统 | 目录：`D:\pythondm\深度学习\code` | 更新：2026-10-15

零、这个项目在做什么

一句话：**输入一段广告文案，输出它属于哪一类违规（或不违规）**，并在模型没把握时转人工复核。

技术上它是一条完整的**文本分类流水线**，同一份数据喂给四套方案横向对比，再叠加模型压缩（量化 / 蒸馏 / 剪枝），最后通过 Flask 服务 + Streamlit 页面交付。

数据来源	样本量	类别数
ChineseHarm-bench	**9,503 条（去重后）**	**6 类**

最佳方案
BERT 微调 F1 0.934

六类标签

ID	类别	含义
0	不违规	正常内容，唯一不需要拦截的类别
1	黑产广告	黑灰产引流、违规推广
2	欺诈	诈骗话术、虚假诱导
3	博彩	赌博引流、赌盘推广
4	谩骂引战	辱骂、挑拨对立
5	低俗色情	色情低俗、擦边内容

一、目录总览

目录	角色	作用	文件数
`01-data`	数据层	存放全部数据集、词典、预训练模型、训练产物	20+
`02-rf`	基线方案 A	TF-IDF + 随机森林	8
`03-fasttext`	基线方案 B	fastText 字级 / 词级四种配置	9
`04_Bett`	**主线方案**	BERT 微调 + 量化 + 蒸馏 + 剪枝，含实验报告	13
`05-LLM`	对照方案	DeepSeek 大模型提示词分类	6
`00-test / test`	**无关**	Flask 入门练习，与本项目无关，`test` 是 `00-test` 的旧副本	7

二、理解全部代码的关键：数据契约

四套方案能共用同一份原始数据，靠的是两条极简约定。理解了它们，其余代码基本是复制粘贴。

契约一：数据文件格式

```
01-data/train.txt      每行：文本 \t 标签索引
01-data/dev.txt        例：加薇xxx看片\t5
01-data/test.txt
01-data/class.txt      第 i 行 = 标签 ID 为 i 的类别名（行号即 ID）
```

契约二：推理函数签名

```
predict_fun(data: dict) -> data    # data['text'] 进，data['pred_class'] 出
```

四套方案的 `predict_fun` 签名完全一致，所以 Flask 服务只需改一行 `import` 就能换模型。

各方案派生的数据文件

文件	给谁用	由谁生成	格式
`train/dev/test.txt`	BERT / RF	`prepare_adaudit_data.py`	文本\t标签索引
`char_train/dev/test.txt`	fastText 字级	`03-fasttext/data_process.py`	`__label__不 违 规 广 告 ...`
`word_train/dev/test.txt`	fastText 词级	`03-fasttext/data_process.py`	`__label__不违规 广告 投放 ...`
`process_train.csv`	随机森林	`02-rf/data_process.py`	`words,label`（jieba 分词后空格连接）

三、01-data ｜ 数据层

代码脚本

文件	功能
`prepare_adaudit_data.py` 本次新增	**数据准备总脚本**：读取 `_raw/chineseharm/` 下 7 个 json → 去重 → 分层抽样 → 写出 train/dev/test.txt + class.txt。**换数据集时唯一需要重写的脚本**
`data_eda.py`	探索性数据分析：统计标签分布、句子长度分布并绘图。换数据后可直接跑
`config.py`	数据层路径配置。**注意：它还指向已备份的 `train_raw.txt` 等旧新闻文件，属历史遗留**，实际训练路径由各方案自己的 config 决定

数据文件

文件	规模	说明
`train.txt`	7,600	训练集
`dev.txt`	947	验证集（用于选最优轮次）
`test.txt`	956	测试集（所有报告数字都来自这里）
`class.txt`	6 行	类别名，行号即标签 ID
`char_*.txt / word_*.txt`	各 3	fastText 用，由脚本生成，勿手改
`process_train.csv`	—	随机森林用，由脚本生成
`stopwords.txt`	—	中文停用词表，jieba 分词时使用

模型产物：`save_model/`

文件	说明
`bert_model.pth` 当前使用	BERT 微调 + 类别加权模型，accuracy 0.9310 / macro-F1 0.9343。**默认加载的推理模型**
`bert_model_baseline.pth`	加权前的基线模型（F1 0.930），用于回滚对比
`bert_student_model.pth`	蒸馏后的 4 层 Student，F1 0.9230，推理快 3.08x
`bert_student_a05_e3.pth`	蒸馏第一版（alpha=0.5 / 3 轮），效果较差，仅存档
`bert_pruned_model.pth`	50% 稀疏度剪枝模型，F1 0.9305
`bert_quantization_model.bin`	动态量化 int8 模型，F1 0.9146，体积 145MB
`bert_quuantizaiton_model.bin` 过期	**旧拼写 + 旧 10 类新闻模型（10-13 生成），已废弃，建议删除**

其他目录

目录	说明
`bert-base-chinese/`	预训练权重与词表（102M 参数，12 层）。离线可用，蒸馏时用它初始化 Student
`_raw/chineseharm/`	从 HuggingFace 下载的 7 个原始 json（bench + 6 个类别文件，约 1.55w 条）
`_backup_news/`	原 THUCNews 新闻分类的 16 个数据文件备份（改造时挪走，未删除）

四、02-rf ｜ 方案 A：TF-IDF + 随机森林

成绩：accuracy 0.78 / macro-F1 0.78 —— 作为传统机器学习基线，用来衬托 BERT 的价值

文件	功能
`config.py`	路径、超参、API 端口（6000）
`data_process.py`	读 txt → jieba 分词 → 去停用词 → 写出 `process_train.csv`
`rf_train.py`	TF-IDF 向量化 + 随机森林训练，保存 `rf_model.pkl` 与 `tfidf_model.pkl`。 已修 删除了 IDE 误插入的 `from pygments.lexer import words`
`rf_predic_fun.py`	推理函数 `predict_fun(data)`，符合统一推理契约
`rf_test.py`	在验证集上评估：准确率 / 精确率 / 召回率 / F1 / 混淆矩阵
`api_flask_server.py`	Flask 服务，`POST /predict`，端口 6000
`api_test.py`	命令行交互测试：手动输入文本 → 调用接口 → 打印结果
`streamlit_app.py`	前端页面，已改为广告审核文案
`dp.log / rf_train.log` 日志	运行日志，可随时删除

五、03-fasttext ｜ 方案 B：fastText 四种配置

字级 vs 词级 × 默认参数 vs 自动调参，四种组合对比

文件	测试集 F1	功能
`fasttext_char_default.py`	**0.9079**	字级切分，默认超参（dim=32, wordNgrams=2）
`fasttext_word_1_default.py`	0.8724	词级切分（jieba），默认超参
`fasttext_char_2_auto.py`	0.8944	字级 + `autotune` 自动调参
`fasttext_word_2_auto.py`	0.8588	词级 + `autotune` 自动调参

支撑文件

文件	功能
`config.py` 重点改造	路径 + 超参。**关键：内置中文路径兼容层**——fastText 是 C++ 内核，用窄字符接口，无法打开含中文的路径（本项目路径含「深度学习」必然失败），故把读写重定向到 `%TEMP%/ft_ascii_workspace`；另附 numpy 2.x 的 `get_input_matrix` 补丁
`data_process.py`	生成 `char_*`（逐字）与 `word_*`（jieba）两组共 6 个文件，支持中文 `__label__`
`fasttext_predict_fun.py`	推理函数，统一签名
`api_flask_server.py`	Flask 服务，端口 6000
`streamlit_app.py`	前端页面
`ft_*.log` 日志	四个训练脚本的运行日志，可删除

六、04_Bett ｜ 主线方案：BERT 微调 + 三种压缩

成绩：accuracy 0.9310 / macro-F1 0.9343，比随机森林高 15 个点

核心训练与推理

文件	功能
`config.py` 重点改造	**全项目最重要的配置文件。**集中管理：数据路径、6 类标签映射、超参（max_len=64 / epochs=3 / lr=2e-5 / batch_size=8）、置信度阈值（0.60）、类别加权开关、蒸馏与剪枝的全部参数。换业务场景时主要改这里
`bert_train.py` 重点改造	训练主脚本：构建 DataLoader → 训练 → 验证 → 按最优 F1 存盘 → 测试集评估 → 输出分类报告与混淆矩阵 → 画训练曲线。 已修 修复 `train()` 无返回值导致曲线画不出、测试集 `drop_last` 丢样本等问题
`bert_predict_fun.py` 重点改造	推理函数。除返回类别外，额外计算 **softmax 置信度**，低于 0.60 返回「需人工复核」并置 `need_review=True` ——这是审核业务特有的拒判机制
`api_flask_server.py`	Flask 服务，端口 6000。`jsonify(data)` 直接透传，所以新增字段无需改动此文件
`streamlit_app.py` 改造	前端页面：展示预测类别、置信度百分比、是否需人工复核、推理耗时

三种模型压缩 本次新增

文件	核心指标	功能
`bert_quantization.py`	390MB → 146MB	动态量化（`quantize_dynamic`，Linear 转 int8），附带体积与延迟核算。 已修 强制 CPU 运行——修复 device 自动识别 CUDA 后出现的设备撞车
`bert_distillation.py`	102M → 45.6M	知识蒸馏：从 Teacher 12 层中等间隔抽 4 层初始化 Student，用 KL 软标签 + 硬标签 CE 联合训练。推理快 3.08x，F1 0.9230
`bert_pruning.py`	50% 稀疏度	全局 L1 非结构化剪枝，分 3 步渐进、每步剪完微调 1 轮，末尾再叠加量化。 已修 弃用 `prune.global_unstructured` 改手动掩码，解决显存缓冲膨胀导致的速度崩溃

产出与日志

文件	说明
实验报告.html 本次新增	完整实验报告（九节）：任务与数据、核心指标、四方案对比、加权实验、混淆矩阵、阈值扫描、量化、蒸馏、剪枝、工程修复清单
train_curves.png	训练曲线图（loss / acc / F1 各三轮）
train_results.json	每轮指标的结构化记录
train.log train_weighted.log quant.log / prune.log distill.log / distill2.log smoke_prune.log / verify_prune.log 日志	各阶段运行日志与调试记录，全部可删除，不影响任何功能

七、05-LLM ｜ 对照方案：DeepSeek 大模型

文件	功能
deepseek_predict_fun.py 重点改造	核心：广告合规审核提示词（6 类定义 + 4 条消歧规则 + 12 条 Few-Shot + 拒判机制）+ 推理函数。用于回答「这个场景值不值得上大模型」
提示词工程测试.py	提示词调试脚本，单条文本快速试效果
测试deepseek接口文档.py	官网接口连通性的最小测试样例
api_flask_server.py	Flask 服务，**端口 5002**（硬编码，与前三个方案的 6000 不同）
streamlit_app.py	前端页面，请求 5002
.env 敏感	**存放 DEEPSEEK_API_KEY。分享项目前务必删除此文件**

八、00-test / test ｜ Flask 入门练习（与本项目无关）

文件	功能
`01_flask入门.py`	Flask 最小示例：定义路由返回字符串
`01_flask 入门_优化版.py`	进阶：模板渲染、表单提交、图片展示
`templates/*.html`	三个 Jinja2 模板：index / welcome / show_image
`imgs/*.jpg`	模板里展示的示例图片

> **test 目录是 00-test 的早期副本**（少一张 img2.jpg，优化版代码略有差异），属于练手时留下的冗余。两个目录都可以安全删除，不影响主线任何功能。

九、运行顺序

1. 准备数据（只在换数据集时需要）

```
cd 01-data
python prepare_adaudit_data.py        # 生成 train/dev/test.txt + class.txt
cd ../03-fasttext && python data_process.py   # 生成 char_* / word_*
cd ../02-rf      && python data_process.py    # 生成 process_train.csv
```

2. 训练四套方案

```
cd 02-rf        && python rf_train.py                 # 随机森林   F1 0.78
cd ../03-fasttext && python fasttext_char_default.py    # fastText    F1 0.9079
cd ../04_Bett   && python bert_train.py                # BERT 微调   F1 0.9343
```

3. 模型压缩（可选）

```
python bert_quantization.py     # 动态量化
python bert_distillation.py     # 知识蒸馏 12→4 层
python bert_pruning.py          # 非结构化剪枝 50%
```

4. 起服务与页面（两个终端）

```
# 终端 1：Flask 推理服务
cd 04_Bett
python api_flask_server.py

# 终端 2：Streamlit 前端
python -m streamlit run streamlit_app.py
```

> **必须用 DeepLearning 环境启动**：`D:\anaconda2024\envs\DeepLearning\python.exe`
> 直接敲 `streamlit run` 会用到 base 环境，它没有装 torch，`config.py` 第一行 import 就会报错。

十、四个必须知道的坑

1. 端口冲突

02-rf、03-fasttext、04_Bett 的 API 端口**都是 6000**，同一时间只能起其中一个。想同时对比要手动改各自 `config.py` 里的 `api_port`。05-LLM 是 5002，不冲突。

2. 项目路径不能改

fastText 的 C++ 内核读不了中文路径，全靠 `03-fasttext/config.py` 里的兼容层把读写搬到 `%TEMP%`。若把项目挪到别的中文目录，理论上仍可用，但不要绕过这个兼容层直接传路径。

3. batch_size 不要设 16

本机是 RTX 3050 Ti（4GB 显存），且浏览器、微信常驻占用。batch=16 会触发显存换页，速度掉到 1 it/s 以下；设 8 可稳定在 6 it/s。

4. 不要用管道跑训练

`python bert_train.py | grep xxx` 会卡死——tqdm 用 `\r` 刷新进度且从不换行，管道会把所有进度攒成一行堆在内存里。正确做法是重定向到文件：`python bert_train.py > train.log 2>&1`，再用 `tr '\r' '\n' < train.log` 查看。

十一、文件性质速查：哪些可以删

性质	可否删除	清单
核心代码	**不可删**	prepare_adaudit_data.py
方案代码	**不可删**	02-rf/*.py、03-fasttext/*.py、04_Bett/*.py、05-LLM/*.py
数据文件	**不可删**	train/dev/test.txt、class.txt、char_*、word_*、process_train.csv、stopwords.txt、bert-base-chinese/
模型产物	**不可删**	save_model/bert_model.pth（其余为对比存档，可按需清理）
日志文件	**可删**	*.log（02-rf 2 个、03-fasttext 4 个、04_Bett 8 个）
过期产物	**建议删**	save_model/bert_quuantizaiton_model.bin（旧拼写 + 旧 10 类模型）
冗余目录	**可删**	test/（00-test 的旧副本）、00-test/（Flask 练手）
敏感文件	**勿外传**	05-LLM/.env（含 API Key）

历史备份	**按需**	01-data/_backup_news/、01-data/_raw/chineseharm/